Pure Virtual Functions and Abstract Base Classes

Theory, Edge Cases & Code Examples

- ✓ Interface Contracts in C++
- ✓ Interface Design
- ✓ Design Principles and Real-World Impact
- ✓ Pure Virtual Functions Can Have Implementations
- ✓ Pure Virtual Destructors Require Definitions

→ *Swipe to learn* →

Pure Virtual Functions and Abstract Base Classes

Theory

📌 1. Pure Virtual Functions - Interface Contracts in C++ (1/2)

Core characteristics:

Aspect	Pure Virtual Function	Regular Virtual Function
Syntax	`virtual void func() = 0;`	`virtual void func() { /* body */ }`
Implementation in base	Optional (can have body defined separately)	Required
Override requirement	**Mandatory** - derived classes must override	Optional - derived classes can override
Makes class abstract	Yes - class cannot be instantiated	No
Purpose	Define interface contracts	Provide customizable behavior
When used	When forcing all derived classes to implement	When providing default behavior that can be customized

Key insight: Despite the `= 0` syntax suggesting "no implementation," pure virtual functions can have a body defined outside the class declaration. This enables shared logic that derived classes can optionally call via `Base::func()` while still enforcing that they must override the function.

Pure Virtual Functions and Abstract Base Classes

Theory

📌 1. Pure Virtual Functions - Interface Contracts in C++ (2/2)

Code example:

```
class Logger {
public:
    virtual void log(const std::string& msg) = 0;  // Pure virtual
    virtual ~Logger() = default;
};

// Pure virtual CAN have a body
void Logger::log(const std::string& msg) {
    std::cout << "[BASE] " << msg << "\n";  // Shared logic
}

class FileLogger : public Logger {
public:
    void log(const std::string& msg) override {
        Logger::log(msg);  // Optionally call base implementation
        std::cout << "[FILE] Writing to disk\n";
    }
};
```

Pure Virtual Functions and Abstract Base Classes

Theory

📌 2. Abstract Base Classes (ABCs) - Interface Design (1/2)

Definition: A class becomes abstract when it contains at least one pure virtual function, making it non-instantiable but usable through pointers/references for polymorphism.

ABC capabilities table:

Feature	Abstract Base Class	Pure Interface (Strict)
Pure virtual functions	At least one	All functions are pure virtual
Regular virtual functions	Can have	Should not have (interface only)
Non-virtual functions	Can have	Should not have
Data members	Can have	Should not have (state-free)
Constructors	Can have (called during derived construction)	Can have (typically protected)
Destructors	Must be virtual	Must be virtual (can be pure)
Instantiation	Cannot instantiate directly	Cannot instantiate directly
Pointers/References	Can create (for polymorphism)	Can create (for polymorphism)

Pure Virtual Functions and Abstract Base Classes

Theory

📌 2. Abstract Base Classes (ABCs) - Interface Design (2/2)

Common ABC usage patterns:

Design Pattern	ABC Role	Example
Strategy	Define algorithm interface	`virtual void execute() = 0;`
Template Method	Define algorithm structure with hooks	Public non-virtual calls private pure virtuals
Abstract Factory	Define object creation interface	`virtual Product* create() = 0;`
Observer	Define update interface	`virtual void notify() = 0;`
Plugin Architecture	Define plugin interface	Runtime loading of implementations

---

Pure Virtual Functions and Abstract Base Classes

Theory

📌 3. Design Principles and Real-World Impact (1/2)

SOLID principles enabled by ABCs:

Principle	How ABCs Enable It	Code Impact
Open/Closed	Add new derived classes without modifying base	New functionality via new classes, not edits
Liskov Substitution	Any derived object works through base pointer	Polymorphic code uses interfaces uniformly
Interface Segregation	Create focused, specific interfaces	Clients depend only on methods they use
Dependency Inversion	High-level code depends on abstractions	Decouples implementation from interface

When to use ABCs:

Scenario	Use ABC	Use Concrete Class
Multiple implementations of same behavior	✅ Define interface	❌ Too rigid
Plugin/module system	✅ Load implementations at runtime	❌ Compile-time only
Testing with mocks	✅ Mock implementations for tests	❌ Hard to mock
Dependency injection	✅ Inject different implementations	❌ Tight coupling
Fixed, single implementation	❌ Unnecessary abstraction	✅ Simpler

Pure Virtual Functions and Abstract Base Classes

Theory

📌 3. Design Principles and Real-World Impact (2/2)

Code example - Dependency Inversion:

```
// ✅ Good: High-level code depends on abstraction
class IDatabase {
public:
    virtual void save(const Data& d) = 0;
    virtual ~IDatabase() = default;
};

class Application {
    IDatabase* db;  // Depends on interface, not concrete class
public:
    Application(IDatabase* database) : db(database) {}
    void processData(const Data& d) {
        db->save(d);  // Works with ANY IDatabase implementation
    }
};

// Can inject MySQL, PostgreSQL, MockDB, etc. at runtime

// ❌ Bad: High-level code depends on concrete implementation
class Application {
    MySQLDatabase db;  // Tightly coupled to MySQL
public:
    void processData(const Data& d) {
        db.save(d);  // Cannot swap database without code changes
    }
};
```

Key takeaway: ABCs are essential for writing flexible, testable, maintainable code that depends on interfaces rather than concrete implementations, enabling runtime polymorphism and adherence to SOLID principles.

Pure Virtual Functions and Abstract Base Classes

Edge Cases

📌 1. Pure Virtual Functions Can Have Implementations

Despite being declared as "pure," a pure virtual function can have a body defined outside the class declaration. This creates an interesting hybrid where the function is still pure (requiring derived classes to override it), but also provides optional default behavior that derived classes can explicitly call.

```
class Logger {
public:
    virtual void log(const std::string& msg) = 0;  // Pure virtual
    virtual ~Logger() = default;
};

// ✅ Pure virtual can have implementation
void Logger::log(const std::string& msg) {
    std::cout << "[BASE LOG] " << msg << "\n";
}

class FileLogger : public Logger {
public:
    void log(const std::string& msg) override {
        Logger::log(msg);  // ✅ Can call base implementation
        std::cout << "[FILE] Writing to disk\n";
    }
};
```

This pattern provides default behavior while enforcing that derived classes must consciously override and acknowledge the function. It's useful for providing common logging, validation, or initialization logic that multiple derived classes can reuse.

Pure Virtual Functions and Abstract Base Classes

Edge Cases

📌 2. Pure Virtual Destructors Require Definitions

A destructor can be declared pure virtual to make a class abstract, but unlike other pure virtual functions, pure virtual destructors must always have a body defined. This is because destructors are called in the destruction chain, and if the base destructor has no implementation, the program will fail to link.

```
class AbstractBase {
public:
    virtual ~AbstractBase() = 0;  // Pure virtual destructor
};

// ❌ Must define it, or linker error
AbstractBase::~AbstractBase() {
    std::cout << "AbstractBase destructor\n";
}

class Concrete : public AbstractBase {
public:
    ~Concrete() override {
        std::cout << "Concrete destructor\n";
    }
};

int main() {
    AbstractBase* ptr = new Concrete();
    delete ptr;  // ✅ Calls both destructors
}
```

This is a common interview trap—declaring a pure virtual destructor without defining it causes cryptic linker errors. The destructor must be defined because it's called during object destruction, even though the class is abstract.

Pure Virtual Functions and Abstract Base Classes

Edge Cases

📌 3. Calling Pure Virtual Functions in Constructors

Calling a pure virtual function from a constructor or destructor is undefined behavior and often causes crashes. During base class construction, the vptr points to the base class's vtable, and if the base class has a pure virtual function, attempting to call it accesses an invalid entry.

```
class Base {
public:
    Base() {
        init();  // ❌ Dangerous: calls pure virtual during construction
    }
    virtual void init() = 0;
};

class Derived : public Base {
public:
    void init() override {
        std::cout << "Derived init\n";
    }
};

int main() {
    Derived d;  // ❌ Undefined behavior or crash
}
```

During `Base()` construction, the object's type is still `Base`, not `Derived`. The vptr points to `Base`'s vtable, which contains a placeholder for the pure virtual function. Calling it results in undefined behavior because there's no valid function to call. Never call virtual functions (especially pure virtual ones) from constructors or destructors.

Pure Virtual Functions and Abstract Base Classes

Edge Cases

📌 4. Intermediate Classes Can Remain Abstract

When inheriting from an abstract base class, intermediate derived classes don't need to override all pure virtual functions. The class remains abstract until some derived class in the hierarchy provides implementations for all pure virtuals.

```
class Interface {
public:
    virtual void operation1() = 0;
    virtual void operation2() = 0;
    virtual ~Interface() = default;
};

class PartialImpl : public Interface {
public:
    void operation1() override {
        std::cout << "operation1 implemented\n";
    }
    // operation2 not overridden - PartialImpl is still abstract
};

class ConcreteImpl : public PartialImpl {
public:
    void operation2() override {
        std::cout << "operation2 implemented\n";
    }
    // ✅ Now all pure virtuals are overridden
};

int main() {
    // PartialImpl p;  // ❌ Error: still abstract
    ConcreteImpl c;    // ✅ OK: all pure virtuals overridden
}
```

This allows for incremental implementation of complex interfaces and is useful in large inheritance hierarchies where different levels provide different pieces of functionality.

Pure Virtual Functions and Abstract Base Classes

Edge Cases

📌 5. Function Hiding vs Pure Virtual Overriding

Adding a function with a different signature in a derived class doesn't override a base pure virtual function—it hides it. This is a common mistake where developers think they're implementing the required interface but are actually hiding it with an overload.

```
class Base {
public:
    virtual void process() = 0;  // Pure virtual with no parameters
};

class Derived : public Base {
public:
    void process(int x) {  // ❌ Different signature - doesn't override
        std::cout << "Derived process\n";
    }
};

int main() {
    // Derived d;  // ❌ Error: Derived is still abstract
}
```

The function `process(int)` is an overload, not an override. The pure virtual `process()` remains unimplemented, so `Derived` is still abstract. Always use the `override` keyword to catch such mistakes at compile-time.

Pure Virtual Functions and Abstract Base Classes

Edge Cases

📌 6. Abstract Classes Can Have Constructors and Data

Abstract classes are not limited to pure virtual functions—they can have constructors, data members, and regular member functions. The constructors are called during derived class construction, allowing the base class to initialize its data.

```
class AbstractShape {
protected:
    std::string name;
    int id;

public:
    AbstractShape(std::string n, int i) : name(n), id(i) {
        std::cout << "AbstractShape constructed\n";
    }

    virtual double area() const = 0;  // Pure virtual

    std::string getName() const { return name; }  // Regular member

    virtual ~AbstractShape() = default;
};

class Circle : public AbstractShape {
    double radius;
public:
    Circle(double r, int id) : AbstractShape("Circle", id), radius(r) {}

    double area() const override {
        return 3.14159 * radius * radius;
    }
};
```

This demonstrates that abstract classes can maintain state and provide partial implementation, not just pure interface definitions. The base class constructor ensures consistent initialization across all derived classes.

Pure Virtual Functions and Abstract Base Classes

Code Examples

📌 1. Basic Abstract Interface

This demonstrates the fundamental pattern of programming to interfaces. The `render` function works with any `IDrawable` implementation, enabling polymorphism and loose coupling.

```
class IDrawable {
public:
    virtual void draw() const = 0;
    virtual ~IDrawable() = default;
};

class Circle : public IDrawable {
    double radius;
public:
    Circle(double r) : radius(r) {}
    void draw() const override {
        std::cout << "Drawing circle with radius " << radius << "\n";
    }
};

class Rectangle : public IDrawable {
    double width, height;
public:
    Rectangle(double w, double h) : width(w), height(h) {}
    void draw() const override {
        std::cout << "Drawing rectangle " << width << "x" << height << "\n";
    }
};

void render(const IDrawable& obj) {  // ✅ Program to interface
    obj.draw();
}
```

Pure Virtual Functions and Abstract Base Classes

Code Examples

📌 2. Pure Virtual Destructor Pattern

Using a pure virtual destructor is a clean way to make a class abstract when you don't have other pure virtual functions. The destructor still needs a body because it's called during destruction.

```
class AbstractResource {
public:
    virtual ~AbstractResource() = 0;  // Makes class abstract via destructor
};

// ✅ Must define the pure virtual destructor
AbstractResource::~AbstractResource() {
    std::cout << "AbstractResource cleanup\n";
}

class FileResource : public AbstractResource {
    std::string filename;
public:
    FileResource(std::string name) : filename(name) {
        std::cout << "Opening file: " << filename << "\n";
    }

    ~FileResource() override {
        std::cout << "Closing file: " << filename << "\n";
    }
};

int main() {
    AbstractResource* res = new FileResource("data.txt");
    delete res;  // ✅ Both destructors called
}
```

Pure Virtual Functions and Abstract Base Classes

Code Examples

📌 3. Template Method Pattern with Pure Virtuals

This demonstrates the Template Method pattern where the base class defines the algorithm structure, mixing required overrides (pure virtual) with optional customization points (regular virtual).

```
class Algorithm {
private:
    virtual void preProcess() {  // ✅ Non-pure with default
        std::cout << "Default preprocessing\n";
    }

    virtual void execute() = 0;  // ✅ Pure virtual - must override

    virtual void postProcess() {  // ✅ Non-pure with default
        std::cout << "Default postprocessing\n";
    }

public:
    void run() {  // ✅ Template method
        preProcess();
        execute();
        postProcess();
    }

    virtual ~Algorithm() = default;
};

class ConcreteAlgorithm : public Algorithm {
private:
    void preProcess() override {
        std::cout << "Custom preprocessing\n";
    }

    void execute() override {
        std::cout << "Main algorithm execution\n";
    }
    // Uses default postProcess
};
```

Pure Virtual Functions and Abstract Base Classes

Code Examples

📌 4. Strategy Pattern with Abstract Interface

The Strategy pattern uses abstract interfaces to allow algorithms to be selected and swapped at runtime, demonstrating dependency injection and the Open/Closed Principle.

```
class SortStrategy {
public:
    virtual void sort(std::vector<int>& data) = 0;
    virtual ~SortStrategy() = default;
};

class BubbleSort : public SortStrategy {
public:
    void sort(std::vector<int>& data) override {
        std::cout << "Sorting with bubble sort\n";
        // Implementation...
    }
};

class QuickSort : public SortStrategy {
public:
    void sort(std::vector<int>& data) override {
        std::cout << "Sorting with quick sort\n";
        // Implementation...
    }
};

class DataProcessor {
    SortStrategy* strategy;
public:
    DataProcessor(SortStrategy* s) : strategy(s) {}

    void process(std::vector<int>& data) {
        std::cout << "Processing data...\n";
        strategy->sort(data);  // ✅ Polymorphic call
    }
};
```

Pure Virtual Functions and Abstract Base Classes

Code Examples

📌 5. Partial Implementation in Abstract Base (1/2)

This shows how abstract classes can provide substantial shared implementation while leaving specific operations to derived classes. The base class maintains state and provides utility methods.

```
class DataStore {
protected:
    std::string connectionString;
    bool connected;

    virtual bool connect() = 0;  // Pure virtual
    virtual void disconnect() = 0;  // Pure virtual

public:
    DataStore(std::string conn) : connectionString(conn), connected(false) {}

    bool isConnected() const { return connected; }  // ✅ Concrete method

    void reconnect() {  // ✅ Concrete method using pure virtuals
        if (connected) disconnect();
        connect();
    }

    virtual ~DataStore() {
        if (connected) disconnect();
    }
};

class SqlDatabase : public DataStore {
public:
    SqlDatabase(std::string conn) : DataStore(conn) {}

    bool connect() override {
        std::cout << "Connecting to SQL: " << connectionString << "\n";
        connected = true;
        return true;
    }

    void disconnect() override {
        std::cout << "Disconnecting from SQL\n";
        connected = false;
// ...continues on next slide
```

Pure Virtual Functions and Abstract Base Classes

Code Examples

📌 5. Partial Implementation in Abstract Base (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }
};
```

Pure Virtual Functions and Abstract Base Classes

Code Examples

📌 6. Pure Virtual with Body for Optional Shared Logic

Providing a body for pure virtual functions allows derived classes to optionally call shared validation or preprocessing logic while still being required to override the function.

```
class Validator {
public:
    virtual bool validate(const std::string& input) = 0;
    virtual ~Validator() = default;
};

// ✅ Provide default implementation
bool Validator::validate(const std::string& input) {
    if (input.empty()) {
        std::cout << "Empty input detected\n";
        return false;
    }
    return true;
}

class EmailValidator : public Validator {
public:
    bool validate(const std::string& input) override {
        if (!Validator::validate(input)) {  // ✅ Call base implementation
            return false;
        }

        // Email-specific validation
        bool hasAt = input.find('@') != std::string::npos;
        std::cout << "Email validation: " << (hasAt ? "valid" : "invalid") << "\n";
        return hasAt;
    }
};
```

Pure Virtual Functions and Abstract Base Classes

Code Examples

📌 7. Multiple Pure Virtual Functions

Interfaces often define multiple related operations that form a cohesive contract. All pure virtual functions must be overridden for the class to be instantiable.

```
class ISerializable {
public:
    virtual std::string serialize() const = 0;
    virtual void deserialize(const std::string& data) = 0;
    virtual ~ISerializable() = default;
};

class Person : public ISerializable {
    std::string name;
    int age;

public:
    Person(std::string n = "", int a = 0) : name(n), age(a) {}

    std::string serialize() const override {
        return name + "," + std::to_string(age);
    }

    void deserialize(const std::string& data) override {
        size_t pos = data.find(',');
        name = data.substr(0, pos);
        age = std::stoi(data.substr(pos + 1));
    }
};

void saveToFile(const ISerializable& obj, const std::string& filename) {
    std::string data = obj.serialize();
    std::cout << "Saving to " << filename << ": " << data << "\n";
}
```

Pure Virtual Functions and Abstract Base Classes

Code Examples

📌 8. Abstract Factory Pattern (1/2)

The Abstract Factory pattern uses abstract base classes to define families of related objects, enabling object creation without specifying concrete classes.

```
class Document {
public:
    virtual void open() = 0;
    virtual void save() = 0;
    virtual ~Document() = default;
};

class PdfDocument : public Document {
public:
    void open() override { std::cout << "Opening PDF\n"; }
    void save() override { std::cout << "Saving PDF\n"; }
};

class WordDocument : public Document {
public:
    void open() override { std::cout << "Opening Word doc\n"; }
    void save() override { std::cout << "Saving Word doc\n"; }
};

class DocumentFactory {
public:
    virtual Document* createDocument() = 0;  // Factory method
    virtual ~DocumentFactory() = default;
};

class PdfFactory : public DocumentFactory {
public:
    Document* createDocument() override {
        return new PdfDocument();
    }
};

class WordFactory : public DocumentFactory {
public:
    Document* createDocument() override {
        return new WordDocument();
// ...continues on next slide
```

Pure Virtual Functions and Abstract Base Classes

Code Examples

📌 8. Abstract Factory Pattern (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }
};
```

What You Learned

Key Takeaways

- ✓ Interface Contracts in C++
- ✓ Interface Design
- ✓ Design Principles and Real-World Impact
- ✓ Pure Virtual Functions Can Have Implementations

→ *Swipe to learn* →